

Aula Passada

- Remoção Lógica e Reuso
 - Abordagem Estática
 - Abordagem Dinâmica Fixa
 - Abordagem dinâmica Variável
 - Fator de Blocagem
 - Algoritmos de Alocação de Registros

Estruturas de Indexação de Dados

Diego Silva Caldeira Rocha

Sumário

- Introdução
- índice
- índice hierárquico
- Árvores de Múltiplos Caminhos
- Árvore B
- Árvore B*
- Árvore B+
- Exercícios

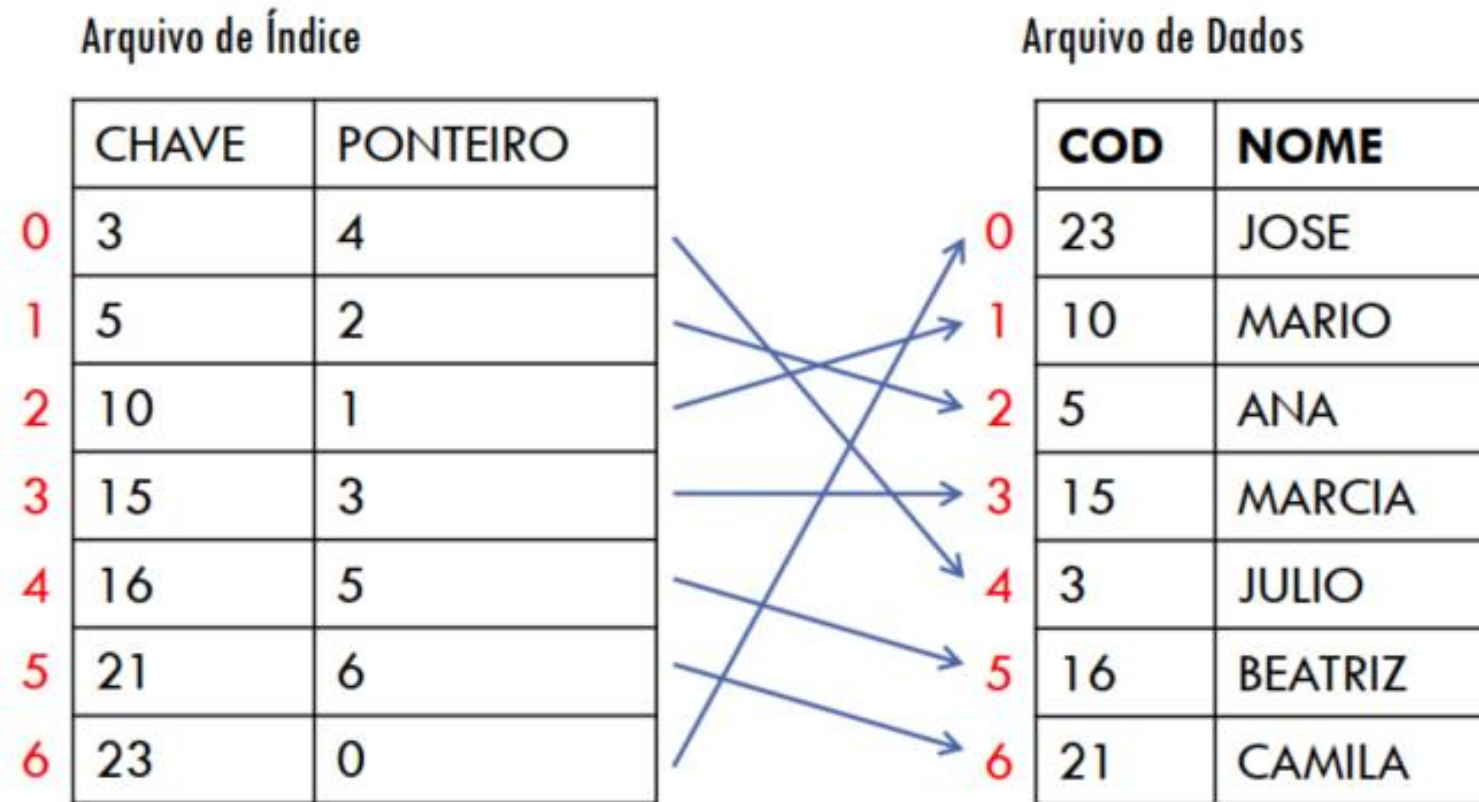
Introdução

- Buscar em arquivo de dados grandes com número elevado de registro.
 - Busca sequencial é muito custosa
 - Se arquivo estiver ordenado pode-se fazer busca binária, mas para arquivos grandes
- É possível acelerar a busca usando duas técnicas:
 - Acesso via cálculo do endereço do registro (*hashing*).
 - Acesso via estrutura de dados auxiliar (índice).

Índice

- Índice é uma estrutura de dados que serve para localizar registros no arquivo de dados
 - Cada entrada do índice contém
 - Valor da chave
 - Ponteiro para o arquivo de dados
 - Pode-se pensar então em dois arquivos:
 - Um de índice
 - Um de dados
- Isso é eficiente?

Exemplo de Índice Plano



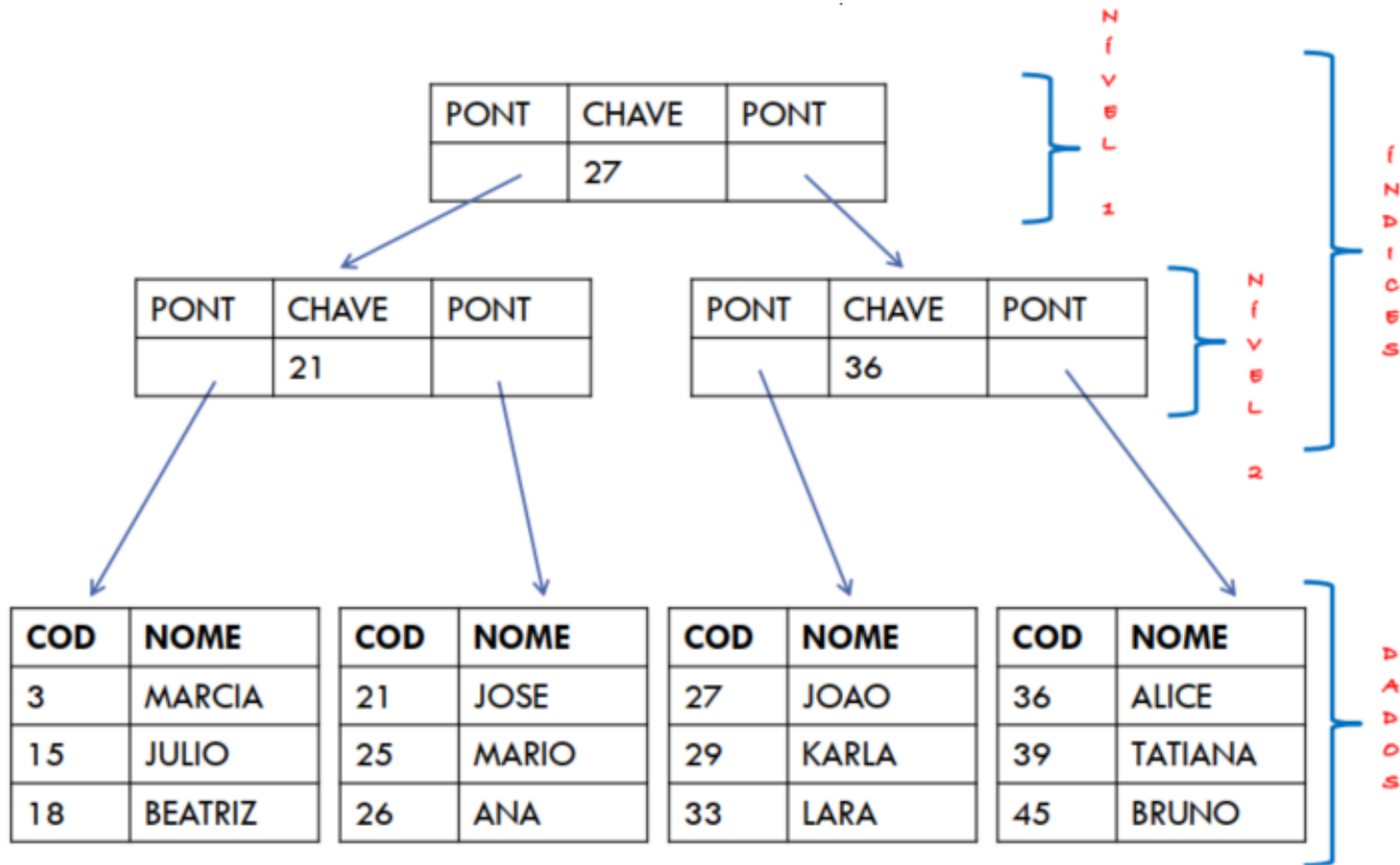
Índice

- Se tivermos que percorrer o arquivo de índice sequencialmente para encontrar uma determinada chave, o índice não terá muita utilidade
 - Pode-se fazer busca um pouco mais eficiente (ex. busca binária), se o arquivo de índice estiver ordenado.
 - Mas mesmo assim não é o ideal.

Solução: Estruturas Hierárquica com os Índices

- Para resolver este problema:
 - Os índices não são estruturas sequenciais, e sim hierárquicas.
 - Os índices não apontam para um registro específico, mas para um bloco de registros (e dentro do bloco é feita busca sequencial) – exige que os registros dentro de um bloco estejam ordenados.

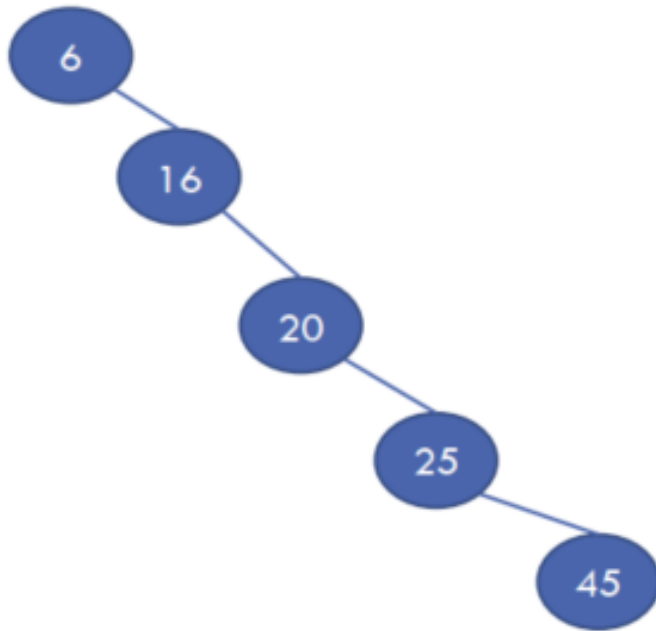
Exemplo de Índice Hierárquico



Hierarquia Lembra Árvore

- A maioria das estruturas de índice é implementada por árvore de busca;
 - Árvore Binária de Busca
 - Árvore AVL
 - Árvore de Múltiplos caminhos

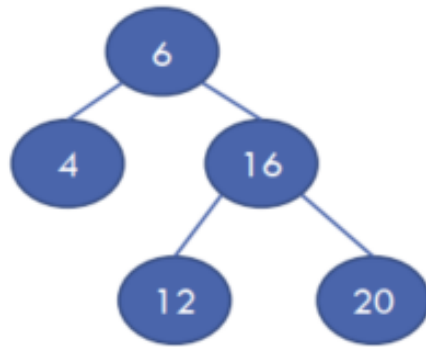
Árvore Binária de Busca



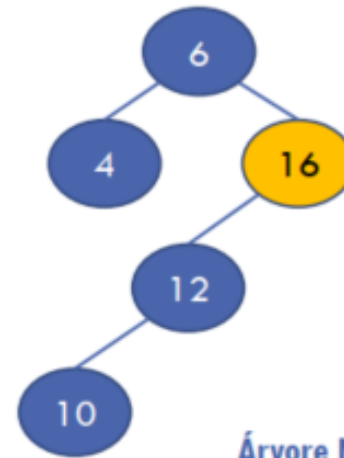
- Altura tende a ser muito grande em relação ao número de nós ou registros que ela contém.
- Se as chaves a serem incluídas estiverem ordenadas, árvore degrada-se rapidamente, tornando-se uma lista encadeada.

Uso de Árvores AVL

- Apesar de serem árvores binárias de busca balanceadas, ainda são excessivamente altas para uso eficiente como estrutura de índice.



Árvore AVL



Árvore Não-AVL

Árvores de Múltiplos Caminhos

- Características
 - Cada nó contém $n-1$ chaves
 - Cada nó contém n filhos
 - As chaves dentro do nó estão ordenadas
 - As chaves dentro do nó funcionam como separadores para os ponteiros para os filhos do nó.
- Exemplos
 - Árvore B
 - Árvore B*
 - Árvore B+

Árvores de Múltiplos Caminhos: Vantagens

- Tem altura bem menor que as árvores binárias
- Ideais para uso como índice de arquivos em disco
- Como as árvores são baixas, são necessários poucos acessos em disco até chegar ao ponteiro para o bloco que contém o registro desejado.

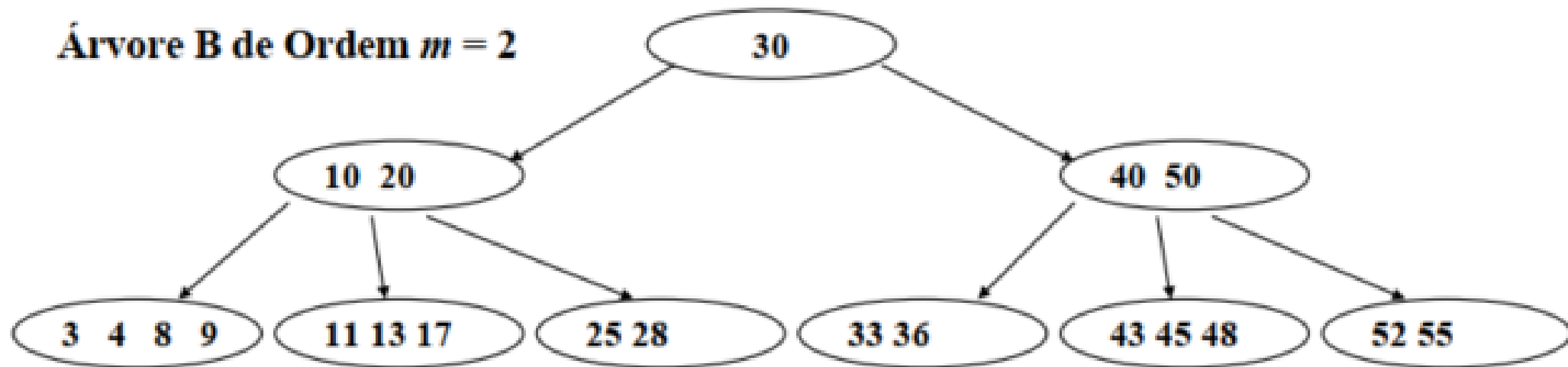
Árvore B

- Bayer e McCreight/72
- Procura reduzir o número de acessos a disco
- Árvore Balanceada de forma heurística
- A árvore 2-3-4 foi uma árvore surgida posteriormente sendo uma estrutura mais rígida.

Árvore B

- Árvore B de Ordem m
 - *Divergência literária, alguns autores falam que ordem é o número máximo de filhos de um nó.*
 - Durante a disciplina AED III, m será metade da capacidade de um nó.
 - Cada pág. contém no mínimo m reg. ($m + 1$ descendentes) e no máximo $2m$ reg. ($2m + 1$ descendentes), exceto a raiz (de 1 a $2m$ registros).
 - Todas as folhas possuem o mesmo nível.

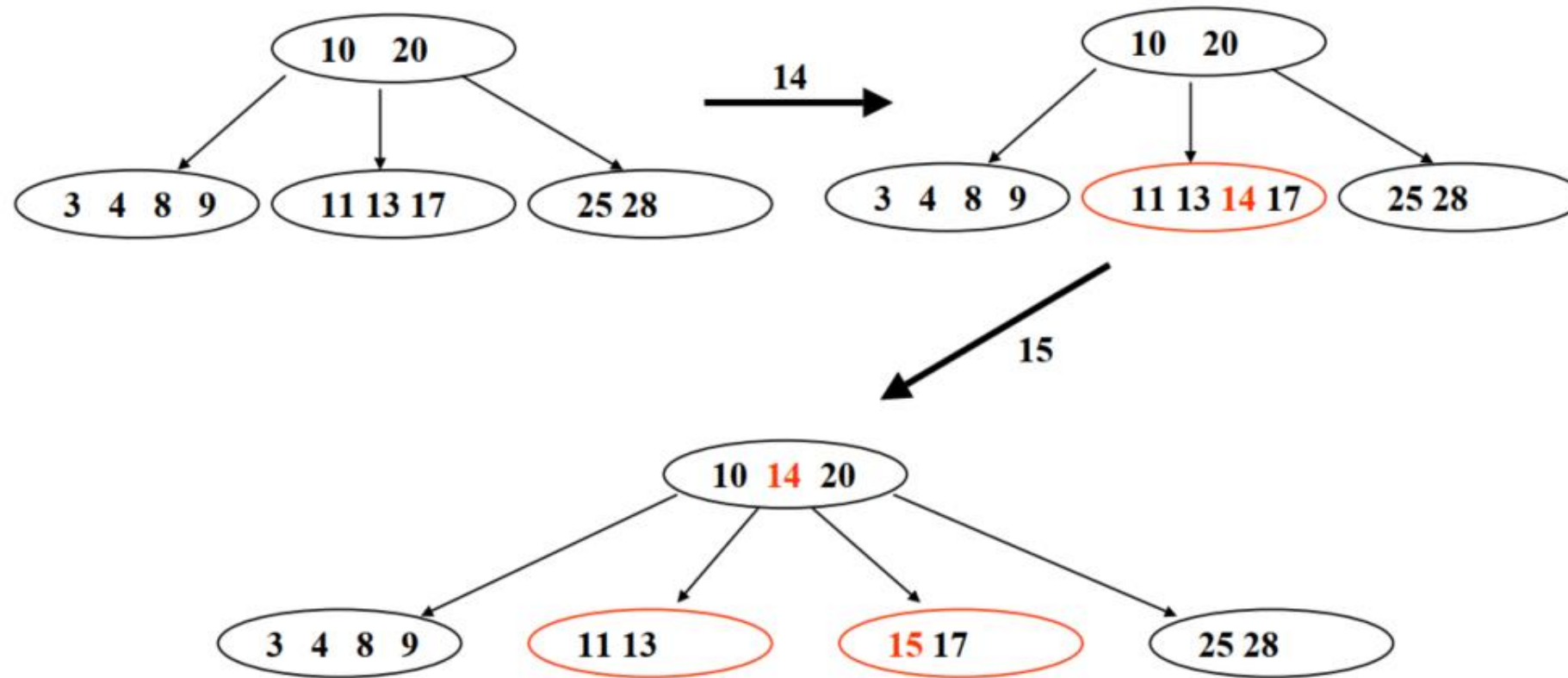
Árvore B ordem $m=2$



Árvore B (Inserção)

- Localizar a página onde o novo registro deve ser inserido;
- Se a quantidade de registros na página for menor que $2m$, inserir o reg. e encerrar;
- Caso contrário, localizar registro entre os $2m+1$, cuja chave tenha valor mediano (incluindo o novo) – “**overflow**”;
- Criar uma nova página, deixando na pág. antiga os registros cujas chaves sejam de valor inferior a mediana e deslocando para a nova aqueles cujas chaves superem a mediana;
- O registro com chave mediana deve ser deslocado para a página superior (ou pai) passando a ocupar uma posição adequada em relação aos já existentes (será necessário reorganizar a página);
- Caso a quantidade de registros da página superior seja $2m$, a inclusão de um novo registro viola a regra de construção da árvore, sendo necessário reaplicar o processo de divisão;
- No pior caso, o processo de divisão se propaga até a página raiz, levando a um aumento da altura da árvore, quando se efetuar a divisão da raiz.

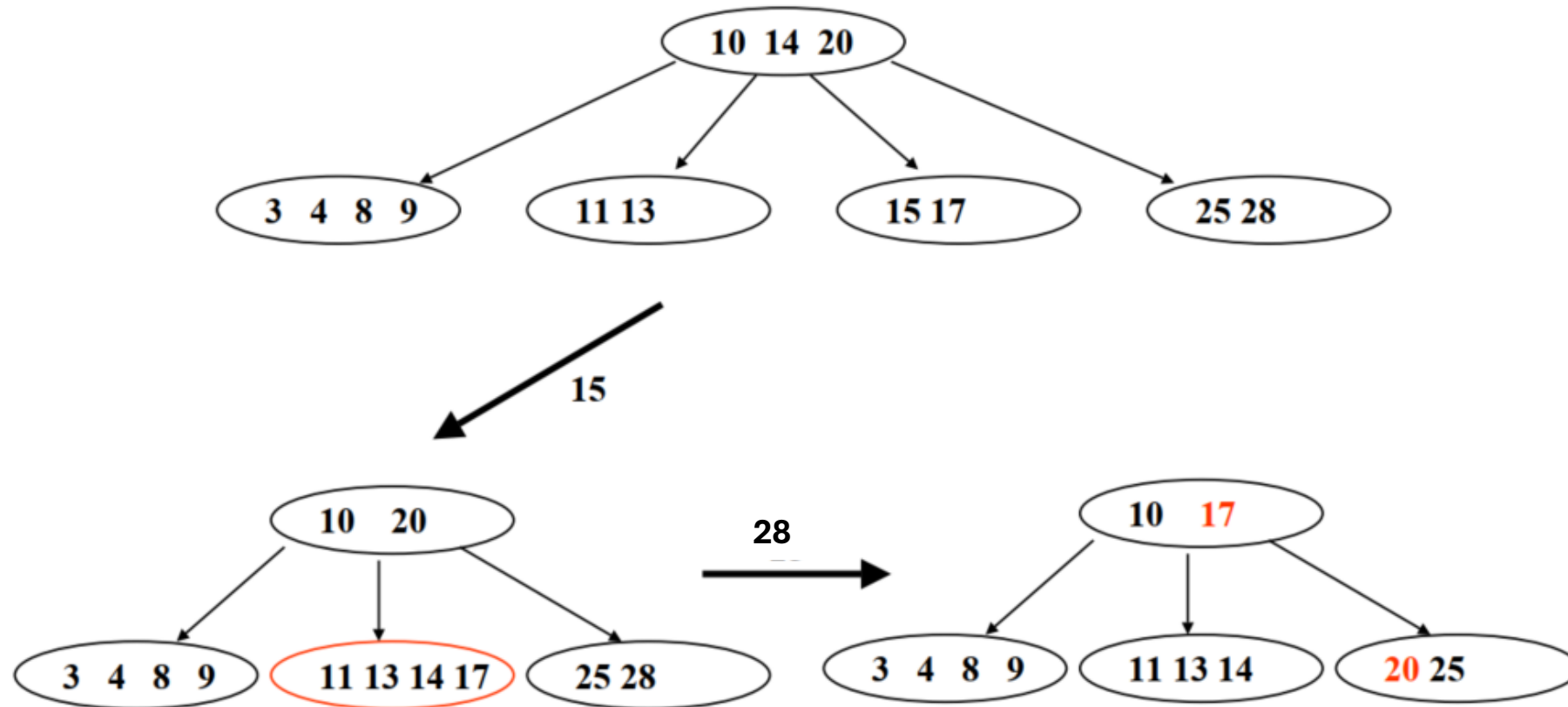
Árvore B (Exemplo de Inserção)



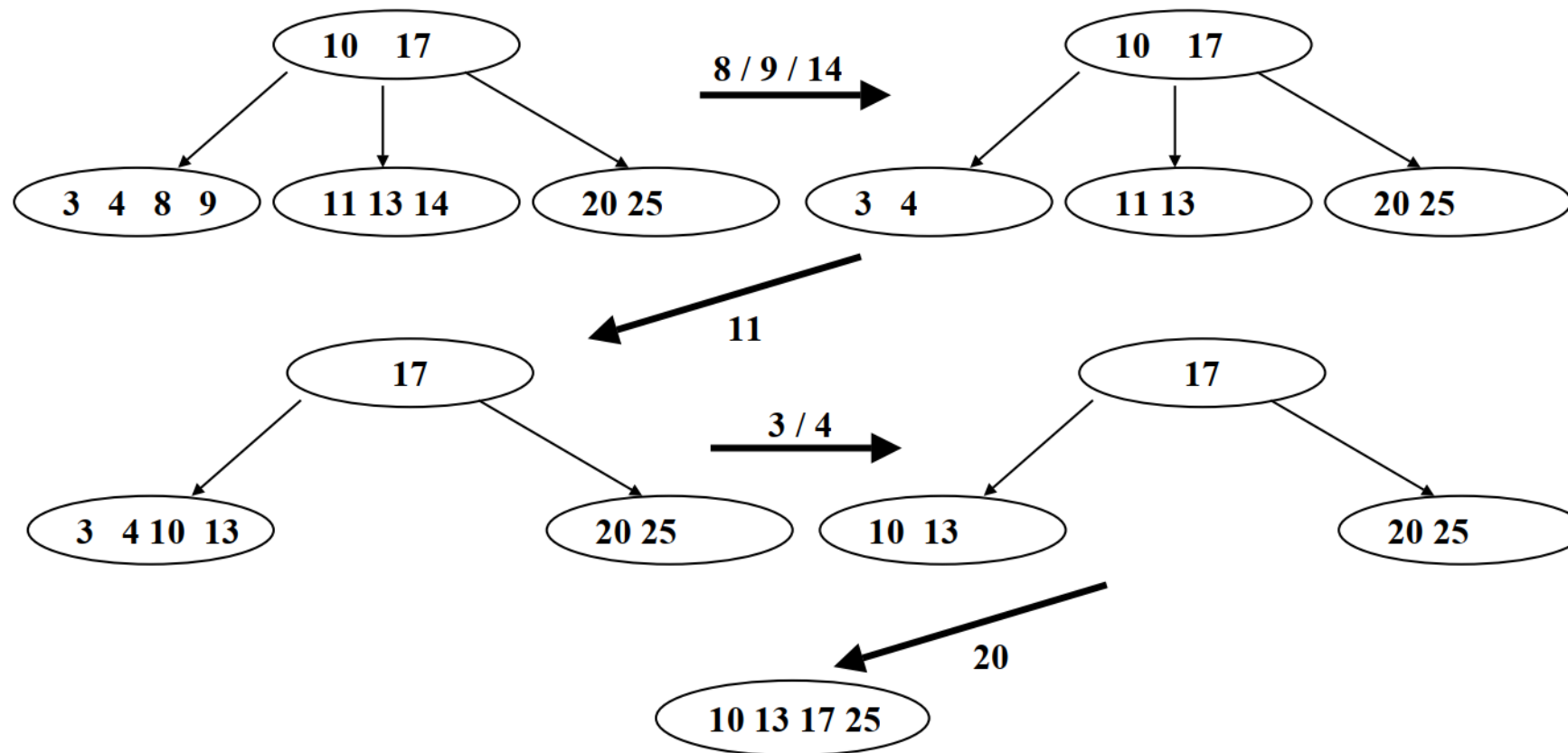
Árvore B (Remoção)

- Remoção de Registro em Página-Folha :
 - Remoção de registro em uma página folha com mais de **m** registros é trivial.
- Caso a página-folha tenha **m** registros – “**underflow**” :
 - Se um de seus irmãos tiver mais de **m** chaves, basta fazer um “empréstimo” com ele, através do nó pai comum a ambos;
 - Se nenhum de seus irmãos tiver mais que **m** chaves, o nó onde ocorreu o “**underflow**” deve ser consolidado com um de seus irmãos, operação que implica na reorganização do nó pai (pois ele perderá a chave que separa os irmãos);
 - Caso o nó pai venha a apresentar um “**underflow**”, reaplicam-se os “empréstimos” e consolidações. No pior caso, o processo de consolidação se propaga até a página raiz, levando a uma redução da altura da árvore.
- Remoção de Registro em Página-Interna : Dificuldade extra no tratamento dos filhos.

Árvore B (Exemplo de Remoção)



Árvore B (Exemplo de Remoção)



Árvore B

Características:

- Manutenção simples / fácil
- Eficiente e bastante versátil
- Custo de pesquisa logarítmico
- No pior caso 50% do espaço reservado pelo arquivo é utilizado pelos dados (a ocupação média aproxima-se de 69%)
- Espaço gasto varia dinamicamente sem a necessidade de reorganizações completas.

Árvore B*

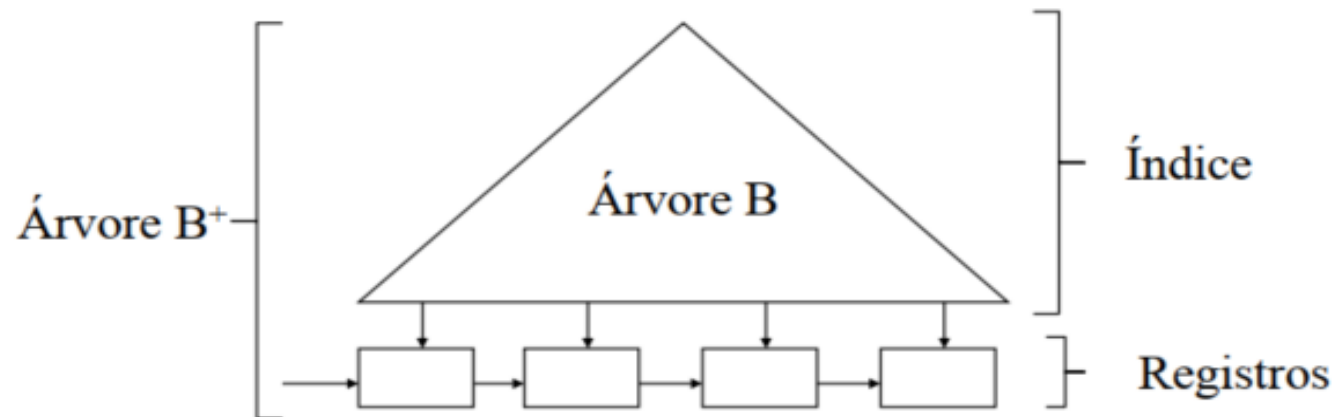
Características:

- Procura retardar a divisão de um nó;
- As chaves do nó, em questão, e as de seu irmão adjacente (além da chave no nó pai que separa os dois), são distribuídas de maneira uniforme.
- Quando ambos estão cheios, os dois nós serão divididos em 3.
- No pior caso cerca de **67%** do espaço reservado pelo arquivo é utilizado pelos dados, chegando a **83%** em média.
- Esta técnica pode ser ampliada, redistribuindo-se as chaves entre todos os irmãos e o pai de um nó completo.
- Impõe um custo extra (acessos adicionais aos irmãos e pai), enquanto que o espaço adicional torna-se cada vez menor.

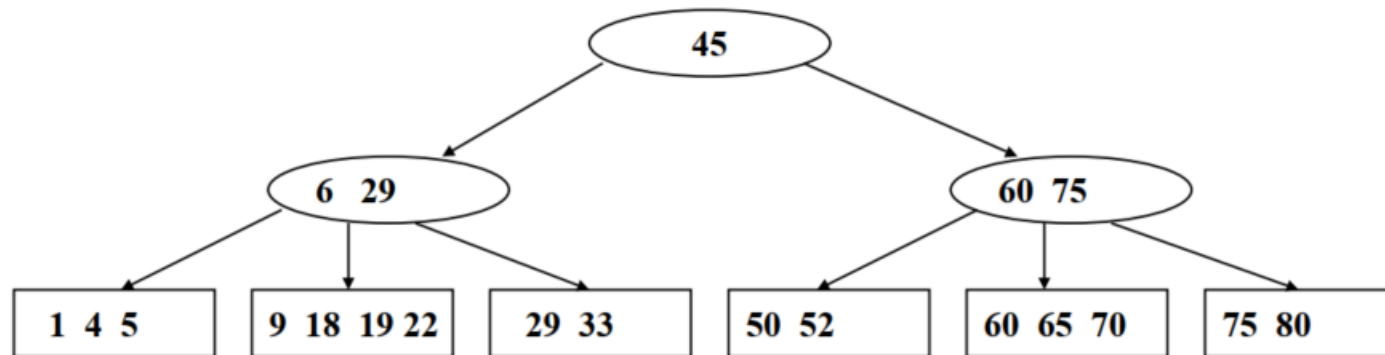
Árvore B+

Carcterísticas:

- Registros só são armazenados nas folhas
- Nós internos contêm apenas chaves
- Páginas folhas são conectadas da esquerda para direita (permitindo acesso sequencial)



Árvore B+



Observações:

- Pesquisa : vai sempre até as folhas (na igualdade caminha a direita)
- Inserção : idêntica a árvore B, só que ao copiar o registro para a página pai também o mantém na folha da direita
- Remoção : simples, pois ocorre sempre nas folhas
- Acesso sequencial facilitado
- Aumento da ordem potencial da árvore

Exercício

- 1) Desenhar uma árvore B (passo a passo) de ordem 3 que contenha as seguintes valores: 8, 1, 6, 3, 14, 36, 32, 43, 39, 41, 38, 4, 5, 42, 2, 7.
- 2) Remova as chaves 14 e 32.

Referência

Szwarcfiter, J.; Markezon, L. Estruturas de Dados e seus Algoritmos, 3a. ed. LTC. Cap. 5

Próxima aula

- Ordenação externa